

PORTADA

Arquitectura de Software

Semana 15 — Web APIs e Implementación de Servicios Web

Diseñar una API que dure años, con el caso de Stripe y la ruptura de la API de Twitter (2012) · Universidad de Antioquia · Ingeniería de Sistemas · 2554608 · 2026-2



Una API que no ha roto la compatibilidad con nadie desde 2011.

Stripe — cada cambio incompatible se resuelve versionando por fecha, nunca rompiendo lo que ya funciona.



TRANSICIÓN

Del esquema en el papel a la API que corre en producción

En las dos sesiones anteriores vimos los principios de SOA y los estándares para describir un servicio (SOAP/WSDL, REST, GraphQL). Hoy damos el siguiente paso: **¿qué decisiones concretas de implementación separan una Web API que se usa con confianza durante quince años de una que rompe a sus clientes cada pocos meses?**

El caso de Stripe, un procesador de pagos donde la confiabilidad de la API no es opcional, es el hilo conductor de la primera mitad de la sesión.

CASO · FICHA TÉCNICA

La API de Stripe

Empresa	Stripe, Inc. — procesador de pagos en línea, fundada en 2010
Producto	API REST para procesar pagos, suscripciones y transferencias, usada por millones de negocios
Primera versión pública	2011
Reconocimiento	Citada de forma recurrente en la industria como referencia de buen diseño de API — documentación interactiva, consistencia y estabilidad
Política de cambios	Ningún cambio incompatible se aplica a una cuenta existente sin que esa cuenta decida adoptarlo explícitamente

Documentado públicamente en la documentación oficial de la API de Stripe (stripe.com/docs/api) y en su blog de ingeniería.

CASO · VERSIONADO POR FECHA

Cada cuenta, anclada a una versión concreta

Stripe versiona su API con un identificador de fecha (ej. `2024-06-20`), no con un número de versión mayor como `v1` , `v2` .

Cada cuenta de Stripe queda anclada a la versión vigente el día en que se creó, y sigue recibiendo exactamente ese comportamiento aunque Stripe publique decenas de nuevas versiones después.

- Si un desarrollador quiere adoptar cambios nuevos, actualiza la versión de su cuenta de forma explícita y controlada.
- Si no hace nada, su integración sigue funcionando exactamente igual, sin sorpresas, durante años.

El costo lo asume Stripe, manteniendo múltiples versiones vivas a la vez — la alternativa a que el costo de una ruptura lo paguen todos sus clientes de golpe.

CASO · OTRAS PRÁCTICAS DOCUMENTADAS

Idempotencia, webhooks y documentación ejecutable

- **Claves de idempotencia:** cada solicitud que crea un cobro puede llevar una clave única; si la red falla y el cliente reintenta con la misma clave, Stripe garantiza que el cobro no se duplica.
- **Webhooks:** en vez de que el cliente pregunte constantemente "¿ya se procesó el pago?" (polling), Stripe notifica proactivamente al servidor del cliente cuando algo relevante ocurre.
- **Documentación interactiva:** cada endpoint documentado permite ejecutar solicitudes de prueba reales desde el propio navegador, con ejemplos de código en múltiples lenguajes generados automáticamente.

Estos tres conceptos — idempotencia, notificación por eventos y documentación ejecutable — reaparecen formalizados en el resto de la sesión.

TRANSICIÓN

De las prácticas de un caso a la disciplina completa

Lo que Stripe hace bien no es un secreto ni algo exclusivo de los pagos: es la aplicación consistente de un conjunto de prácticas de diseño de Web APIs bien documentadas en la industria. **Formalicemos esas prácticas una por una.**

CONCEPTO

Diseño de recursos y URIs

Una Web API REST bien diseñada organiza sus URIs alrededor de **recursos** (sustantivos), no de acciones (verbos) — la acción ya la expresa el verbo HTTP (s14 de la sesión anterior).

Bien diseñado	Mal diseñado
GET /pedidos/4521	GET /obtenerPedido?id=4521
POST /pedidos	POST /crearPedido
GET /clientes/88/pedidos	GET /pedidosDeUnCliente?cliente=88

Sustantivos en plural, anidación clara para relaciones (cliente → sus pedidos), y el verbo HTTP como única fuente de la acción.

CONCEPTO · DE VUELTA AL CASO

Estrategias de versionado

Estrategia	Ejemplo	Trade-off
En la URI	/v1/pedidos	Simple y visible, pero mezcla la versión con la identidad del recurso.
En un header	Accept: application/vnd.api+json;version=2	URIs limpias, pero menos visible/debuggeable a simple vista.
Por fecha, por cuenta	Stripe-Version: 2024-06-20	Máxima estabilidad para el cliente (s05), mayor complejidad operativa para el proveedor.

No existe una estrategia universalmente "correcta" — es, otra vez, un trade-off arquitectónico entre simplicidad para el proveedor y estabilidad para el consumidor.

CONCEPTO

Paginación: offset vs. cursor

Cuando un recurso puede devolver miles de elementos (ej. todas las transacciones de una cuenta), la API debe paginar la respuesta en vez de devolverlos todos de una vez.

- **Paginación por offset:** `?page=3&limit=20` — simple de implementar, pero puede saltarse o repetir elementos si la tabla cambia entre una página y otra.
- **Paginación por cursor:** `?cursor=eyJpZCI6NDUyMX0&limit=20` — el cursor apunta a una posición estable en los datos (ej. el último ID visto); más robusta ante datos que cambian en tiempo real, usada por Stripe y por la mayoría de las APIs de alto tráfico.

CONCEPTO

Autenticación y autorización

Mecanismo	Cómo funciona	Uso típico
API key	Un token fijo enviado en cada solicitud (header o query)	Integraciones servidor-a-servidor simples
OAuth 2.0	Protocolo de autorización delegada — el usuario aprueba que una app acceda a sus datos, sin compartir su contraseña	Apps de terceros que actúan en nombre de un usuario
JWT (JSON Web Token)	Token firmado que contiene la identidad y permisos del usuario, verificable sin consultar una base de datos central	Autenticación stateless entre microservicios

Autenticación responde "¿quién eres?"; **autorización** responde "¿qué puedes hacer?" — son conceptos distintos que a menudo se confunden.

CONCEPTO

Rate limiting: proteger el servicio de sus propios clientes

El **rate limiting (limitación de tasa)** restringe cuántas solicitudes puede hacer un cliente en una ventana de tiempo, para proteger la disponibilidad del servicio ante uso excesivo, accidental o malicioso.

- Se comunica normalmente con headers de respuesta: `X-RateLimit-Limit` , `X-RateLimit-Remaining` , `X-RateLimit-Reset` .
- Al excederse, el servidor responde `429 Too Many Requests` .

El rate limiting reaparece en la segunda mitad de esta sesión: fue una de las herramientas que Twitter usó — de forma mucho más agresiva — en el caso que viene a continuación.

CONCEPTO · DE VUELTA AL CASO

Idempotencia, formalizada

Una operación es idempotente cuando ejecutarla varias veces produce exactamente el mismo resultado que ejecutarla una sola vez. GET, PUT y DELETE son idempotentes por definición del estándar HTTP; POST no lo es — cada solicitud POST puede crear un nuevo recurso.

Por eso Stripe (s06) agrega **claves de idempotencia** explícitas para operaciones POST sensibles como crear un cobro: si la conexión falla justo después de que el cobro se procesó y el cliente reintenta automáticamente, la clave le dice al servidor "esta ya la ejecuté, no la repitas".

Sin idempotencia, un simple reintento de red ante una falla temporal puede convertirse en un cobro duplicado a un cliente real.

CONCEPTO

Documentar la API: OpenAPI y la documentación ejecutable

OpenAPI (antes Swagger) es un estándar para describir formalmente una **API REST** — sus recursos, operaciones, parámetros y respuestas — en un documento YAML o JSON legible tanto por humanos como por herramientas.

- Genera documentación interactiva navegable automáticamente.
- Permite generar clientes (SDKs) en distintos lenguajes de forma automática.
- Es, para REST, un equivalente funcional al WSDL de SOAP (s10 de la sesión anterior) — mismo propósito, formato distinto.

CONTRAEJEMPLO · FICHA TÉCNICA

La API de Twitter, versión 1.1 — 2012

Empresa	Twitter, Inc.
Anuncio	Agosto de 2012, publicado por Michael Sippey , entonces director de plataforma de Twitter, en el blog oficial de desarrolladores
Cambio	Deprecación de la API v1.0 y migración forzada a la v1.1
Restricciones nuevas	Límites de tasa mucho más estrictos, autenticación OAuth obligatoria en todas las llamadas, y "Lineamientos de Visualización" que restringían cómo las apps de terceros podían mostrar tuits
Impacto reportado	Varias aplicaciones de terceros conocidas de esa época (clientes alternativos de Twitter) quedaron inviables comercialmente bajo las nuevas reglas

Ampliamente cubierto en su momento por medios especializados como TechCrunch y The Verge, y documentado en el propio blog de desarrolladores de Twitter.

CONTRAEJEMPLO · LO QUE SALIÓ MAL

Una migración técnicamente anunciada, estratégicamente abrupta

Twitter sí publicó documentación y un plazo de migración de la v1.0 a la v1.1 — no fue un corte sin aviso. El problema no fue solo técnico: fue que **las nuevas reglas de negocio (límites de tasa mucho más bajos para apps de terceros, restricciones de cuántos usuarios podía tener un cliente alternativo) hacían inviable seguir operando** muchas de esas aplicaciones, sin importar cuán bien migraran su código.

La lección arquitectónica no es solo "avisa con tiempo" — es que un cambio de API también comunica, implícita o explícitamente, quién puede seguir construyendo sobre tu plataforma y quién no.

TENSIÓN

Stripe vs. Twitter: mismo problema, decisiones opuestas

	Stripe	Twitter (2012)
Ante un cambio incompatible	Se crea una nueva versión; las cuentas existentes no se mueven a menos que lo pidan	Se deprecó la versión anterior con fecha límite obligatoria para todos
Quién asume el costo del cambio	Stripe, manteniendo varias versiones vivas	Los desarrolladores externos, migrando bajo nuevas restricciones de negocio
Efecto en el ecosistema de terceros	Confianza sostenida durante más de una década	Contracción documentada del ecosistema de apps de terceros

No es solo una diferencia técnica de versionado: es una diferencia de a quién prioriza la arquitectura de la API cuando hay que elegir.

CONCEPTO

Cómo deprecar una API sin repetir el caso de Twitter

- **Anunciar con mucha anticipación** — meses, no semanas — y en canales que los desarrolladores realmente monitorean.
- **Header `Sunset` o `Deprecation`** : la propia respuesta HTTP de la versión antigua indica la fecha exacta en que dejará de funcionar.
- **Mantener ambas versiones vivas** durante el período de transición, en vez de forzar un corte.
- **Documentar la guía de migración** campo por campo, endpoint por endpoint — no solo "lean la v2".

Estas prácticas no eliminan el costo de deprecar una API, pero lo distribuyen de forma predecible en vez de imponerlo de golpe.

CONCEPTO EN DISPUTA

¿De quién es la API: del proveedor o del ecosistema que la usa?

Es tentador pensar que una API le pertenece por completo a quien la construye, y que puede cambiarla como quiera. El caso de Twitter (s15-s17) muestra que esa visión tiene un costo real: **un ecosistema de terceros que invirtió tiempo y dinero construyendo sobre esa API también tiene, en la práctica, un interés legítimo en su estabilidad** — aunque no tenga poder formal de decisión sobre ella.

No hay una respuesta única correcta: Stripe optó por absorber el costo de la estabilidad; Twitter optó por priorizar el control de su plataforma. Ambas son decisiones arquitectónicas y de negocio válidas, con consecuencias distintas para quienes dependen de la API.

INTERACCIÓN

Diseñen el versionado de su propia API

En equipos, para la API de su proyecto CodeF@ctory:

1. ¿Qué estrategia de versionado (s09) usarían: en la URI, en un header, o por fecha como Stripe? Justifiquen con el trade-off correspondiente.
2. Identifiquen una operación de su API que debería ser idempotente (s13, ej. crear un pago o una matrícula) y qué pasaría si no lo fuera.
3. Si tuvieran que deprecarse un endpoint hoy, ¿qué pasos de s18 aplicarían primero?

5 minutos · respondan en voz alta o en el chat

SÍNTESIS

Ideas clave de hoy

1. Diseñar recursos y URIs alrededor de sustantivos, no de acciones — el verbo HTTP ya expresa la acción.
2. Versionar (URI, header o por fecha como Stripe), paginar con cursor en sistemas de alto tráfico, y limitar la tasa de solicitudes (rate limiting) son decisiones de diseño explícitas, no detalles de implementación.
3. Autenticación (quién eres) y autorización (qué puedes hacer) son conceptos distintos; API keys, OAuth 2.0 y JWT resuelven necesidades diferentes.
4. La **idempotencia** protege contra reintentos de red duplicando operaciones sensibles; **OpenAPI** documenta la API de forma ejecutable, como el WSDL lo hacía para SOAP.
5. El caso de Twitter (2012) frente al de Stripe muestra que deprecitar una API no es solo un problema técnico: es una decisión sobre a quién protege la arquitectura cuando algo tiene que romperse.

CIERRE

Para la próxima semana

- Repasar las estrategias de versionado (s09) y las prácticas de deprecación responsable (s18) — se retoman al diseñar el versionado de servicios en microservicios.
- Revisar si su API de CodeF@ctory ya implementa autenticación (s11) y manejo de errores con los códigos HTTP correctos.

La próxima sesión cierra la Unidad 4 con Arquitecturas Cloud Native, Microservicios (DDD, CQRS, Event Sourcing, API Gateway) y Serverless (FaaS) — cómo estas mismas Web APIs se organizan a la escala de un sistema completo.

